PC Build Store

Object Oriented Programming

Project Report

Group Members

- Abdulhanan - Dashboard, Reports, AI Chat
- Ahmad Mushtaq (2501803) - Billing Module
- Aitzaz (2501805) - Build Upgrades
- Haroon - Build Configurator

Object Oriented Programming - Final Submission

2025-2026

Table of Contents

1. Introduction

PC Build Store is a desktop application designed to help users configure, compare, and purchase custom PC builds. The application provides a comprehensive catalog of 78+ components across 6 categories (CPU, GPU, RAM, Storage, PSU, Motherboard) with real pricing data from the Pakistani market.

The system allows users to browse and search PC components by category, build custom PCs with real-time compatibility filtering, upgrade existing builds with smart compatibility checks, purchase builds and receive printable/saveable receipts, and get AI-powered build suggestions through an integrated chat bot.

The application is built using Java 17 with Swing GUI and MariaDB for data persistence, following object-oriented design principles throughout.

Tech Stack
Java 17 | Swing GUI | MariaDB 10.4 | JDBC | NetBeans Ant | KoboldCPP AI

2. Problem Statement

Building a custom PC is a complex task that requires knowledge of hardware compatibility, pricing, and performance metrics. Users often struggle with:

- Identifying compatible components across different categories
- Comparing prices from multiple vendors
- Understanding technical specifications (socket types, DDR generations, power requirements)
- Getting expert advice on optimal configurations

Our solution addresses these challenges by providing an intelligent PC configurator with automated compatibility checks, real-time pricing, and AI-powered recommendations.

3. Objectives

Primary Objectives

- Build a user-friendly PC configurator with drag-and-drop simplicity
- Implement automated compatibility checking across all component types
- Provide real-time pricing from Pakistani vendors (zahcomputers.pk, galaxy.pk)
- Integrate AI chat assistance for build recommendations
- Generate professional receipts for purchase records

Secondary Objectives

- Implement a dark theme UI for modern aesthetics
- Cache product images for fast loading
- Support streaming AI responses for interactive chat
- Generate revenue and performance analytics reports

- Provide modular architecture for easy maintenance

4. System Architecture

The application follows a layered MVC architecture:

Data Flow

User Interface -> DAO Layer -> JDBC -> MariaDB -> DAO -> UI Update

5. Module Distribution

Module	Lead	GUI Work	Backend	Database
Dashboard	Abdulhanan	Hero, Stats, Chat	Chat Streaming	Stats Queries
Build Config	Haroon	Cart, Grid, Tabs	Part Filtering	Part Queries
Build Upgrades	Aitzaz	Upgrade UI	Compatibility	Build Updates
Billing	Ahmad	Receipt View	Print/Save	Bill Inserts
Reports	Abdulhanan	Charts	Aggregation	Report Queries
AI Chat	Abdulhanan	Settings Dlg	SSE Streaming	N/A

6. Integration Strategy

Integration was performed in 4 phases:

Phase 1 - Database Setup

Created MariaDB schema with 6 tables, populated 78+ components with real pricing data, established JDBC connection pool.

Phase 2 - Module Development

Each team member developed their assigned module independently using consistent theme (Theme.java) and shared DAO layer.

Phase 3 - Cross-Module Integration

Dashboard links to all modules, Build Catalog passes builds to Upgrades, Billing receives builds for purchase, Reports aggregate data.

Phase 4 - Feature Completion

AI chat integration with streaming responses, image caching, receipt print/save, add part dialog.

7. Implementation Details

7.1 Dashboard Module (Abdulhanan)

- Hero banner with gradient background
- Statistics cards (total builds, parts, revenue, purchases)
- Featured parts grid with lazy-loaded images
- AI chat panel with streaming SSE responses
- Navigation buttons to all modules

Key: GridBagLayout for responsive stat cards, FontMetrics-based chat bubble heights, KoboldCPP integration via OpenAI-compatible API.

7.2 Build Configurator (Haroon)

- 6-slot PC builder (CPU, GPU, RAM, Storage, PSU, Motherboard)
- Category tabs for filtering components
- Product grid with images and specs
- Cart sidebar showing selected parts
- Real-time total price and performance score

Key: Motherboard compatibility filtering, DDR generation matching, cart state management, ImageCache parallel fetching.

7.3 Build Upgrades (Aitzaz)

- Loading existing builds for modification
- 6-slot upgrade interface with current parts
- Compatibility checking (socket, DDR, wattage)
- Apply upgrades button to save changes

Key: Socket compatibility matrix, DDR validation, wattage calculation, build part replacement with score recalculation.

7.4 Billing Module (Ahmad)

- Build summary display with all selected parts
- Purchase processing with database storage
- Receipt view with formatted HTML output
- Print receipt via PrinterJob
- Save receipt as HTML via JFileChooser

Key: Printable interface, HTML receipt generation, JFileChooser with null parent to avoid modal blocking.

7.5 Reports Module (Abdulhanan)

- Revenue over time chart (line chart)
- Parts distribution by category (pie chart)
- Performance metrics (bar chart)
- Summary statistics cards

Key: Custom JPanel painting for chart rendering, date-based revenue aggregation, category-wise counting.

7.6 AI Chat (Abdulhanan)

- OpenAI-compatible API integration
- Streaming SSE response parsing
- KoboldCPP backend support (Qwen3 model)
- Configurable settings dialog

Key: java.net.http.HttpClient for streaming, SSE parsing, dual-field JSON (content + reasoning_content), settings persistence.

8. Database Design

Table: categories

category_id INT PK
name VARCHAR(50)

Table: parts

part_id INT PK
category_id INT FK
brand VARCHAR(50)
name VARCHAR(100)
price INT
performance_score INT
socket_type VARCHAR(20)
ddr_generation INT
core_count INT
clock_speed DOUBLE
vram INT
memory_speed INT
capacity INT
read_speed INT
wattage INT
efficiency VARCHAR(10)
image_path VARCHAR(500)

Table: builds

build_id INT PK
name VARCHAR(100)
total_price INT
total_score INT

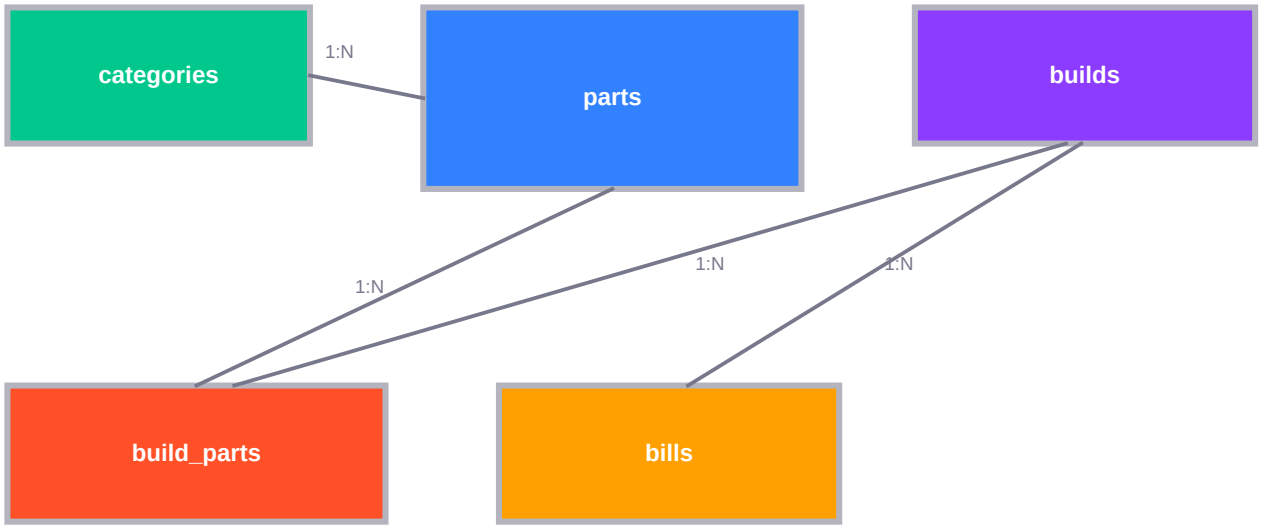
Table: build_parts

build_id INT FK+PK
category_id INT FK+PK
part_id INT FK
price_at_add INT

Table: bills

bill_id INT PK
build_id INT FK
final_price INT
final_score INT
purchase_date TIMESTAMP

8B. Entity-Relationship Diagram



9. Testing

Test: Dashboard

Stats cards display correct counts, navigation buttons work, chat streaming functional, featured parts load.

Test: Build Configurator

Category tabs switch correctly, 6-slot selection works, price/score calculate in real-time, cart updates.

Test: Build Upgrades

Existing builds load, compatibility checks catch mismatches, part replacement works, score recalculates.

Test: Billing

Purchase inserts correctly, receipt displays all parts, print dialog opens, save creates valid HTML file.

Test: Reports

Revenue chart shows correct data, parts distribution accurate, performance metrics display properly.

Test: AI Chat

Streaming responses parse correctly, KoboldCPP connects, settings persist, error handling works.

10. Results

The PC Build Store application successfully delivers:

- Comprehensive catalog of 78+ PC components with real pricing
- Automated compatibility checking across all component types
- Real-time build cost and performance calculation
- Professional receipt generation with print/save options
- AI-powered build assistance via streaming chat
- Dark theme UI with modern aesthetics

11. Conclusion

The PC Build Store project demonstrates effective application of object-oriented programming principles including:

Encapsulation	Model classes with private fields and public getters
Inheritance	JPanel anonymous inner classes for custom components
Polymorphism	DAO pattern for interchangeable data access
Abstraction	Separation of concerns across layers
Composition	GUI panels composed of specialized sub-components

The modular architecture allowed parallel development by team members while maintaining code consistency through shared theme and DAO layers.

Key technical achievements include streaming AI integration with KoboldCPP, 3-tier image caching system, real-time compatibility checking, and professional receipt generation.